



Agentic Engineering

Module 2 Project

The Action Agent: Secure Tool Execution with MCP

1 Exercise Overview

In Module 1, you built the brain: a planning agent that thinks, decomposes goals, and produces structured execution plans. But a plan that never acts is just a document.

Module 2 introduces action. You will build an **Action Agent** a governed, Pydantic AI-powered assistant connected to real tools through the Model Context Protocol (MCP). This agent does not simulate behaviour or return static responses. It executes real operations against live data, makes decisions about what actions are permitted, pauses for human approval when required, and maintains a complete audit trail of every action it takes.

This is the transition from reasoning to doing, and with the ability to act, comes the responsibility to govern.

2 Learning Objectives

By the end of this exercise, you should be able to:

- Build a custom MCP server that exposes functional tools to an AI agent.
- Connect a Pydantic AI agent to an MCP server and execute tool calls.
- Implement Role-Based Access Control (RBAC) to restrict agent actions by user role.
- Integrate a Human-in-the-Loop (HITL) approval gate for sensitive or irreversible operations.
- Apply PII masking to prevent sensitive data from reaching the LLM.
- Produce structured audit logs that record every tool call, decision, and error.
- Work with live, dynamic data from a database rather than static files.

3 Scenario

Real-world AI assistants do not just answer questions, they act on behalf of their users. A student portal assistant books appointments, checks grades, and submits forms. An IT support agent resets passwords, provisions access, and logs incidents. An inventory system agent places orders, updates stock levels, and flags discrepancies.

In every case, the ability to act introduces risk. Actions can be irreversible. Data can be sensitive. Users have different levels of trust. Systems must be accountable.

Your task is to build an AI assistant in a domain of your choice that can take real, governed actions through a secure tool layer, demonstrating that agentic systems can be both capable and safe.

NOTE

This exercise deliberately does not prescribe a single domain. Choose a context that is meaningful to you. The concepts of MCP connectivity, RBAC, HITL, PII masking, and audit logging apply universally.

4 Domain Selection

Select any domain that supports a realistic set of data-driven actions. The following are provided as examples only. You are not required to choose from this list.

Domain	Example Actions	Example Data
Student Portal	Book appointment, check grade, submit leave	Students, courses, schedules
IT Support	Reset password, provision access, log incident	Users, tickets, permissions
Healthcare Chatbot	Book appointment, retrieve test results, flag alerts	Patients, doctors, records
Inventory Management	Update stock, place order, flag discrepancy	Products, suppliers, warehouses
HR Operations	Submit leave, retrieve payslip, update profile	Employees, departments, policies
Custom Domain	Any domain that satisfies all requirements	Your choice

5 Scope: What You Are Building

You are building a governed, action-capable AI assistant with the following components:

Component	Description
MCP Server	A custom server exposing at least 3 functional tools backed by live data
Pydantic AI Agent	An agent that calls MCP tools and returns structured, validated responses
RBAC Layer	Programmatic role enforcement that restricts which tools each user role may invoke
HITL Checkpoint	A pause mechanism that halts execution and awaits human approval for sensitive actions
PII Masking	A pre-processing layer that redacts sensitive fields before data reaches the LLM
Audit Logger	A structured log of every tool call, access decision, and error in the session

Unlike the Module 1 planner, this agent is action-capable. It reads from and writes to a live database, executes real operations, and produces measurable side effects. With that power comes the governance obligations listed above.

6 Technical Stack

6.1 Required Technologies

Component	Technology	Purpose
Language	Python 3.11+	Core programming language
Agent Framework	Pydantic AI	Agent construction and structured outputs
Schema Validation	Pydantic	Validated, typed data models
Tool Protocol	MCP Python SDK	Server-client tool connectivity
Database	SQLite (minimum)	Live, dynamic data source
LLM Provider	OpenAI / Anthropic / Google / Groq	Reasoning and tool-call engine
Package Manager	uv	Fast Python dependency management

DATABASE A flat file (CSV, JSON) is not acceptable as a data source for this project. Your tools must read from and write to a proper database. SQLite is the minimum requirement. PostgreSQL, MySQL, or a cloud-hosted database are also acceptable and recommended for stretch goals.

6.2 Supported LLM Providers

You may use any of the following providers:

- OpenAI (GPT-4o recommended)
- Anthropic (Claude Sonnet or Haiku)
- Google (Gemini 1.5 Pro or Flash)
- Groq (for low-latency local inference)
- Ollama (optional, for fully local setups)

Store all API keys as environment variables. Never hard-code credentials in source files.

7 Functional Requirements

Your implementation must satisfy all requirements below. Each is assessed independently in the grading rubric.

#	Requirement	Description
1	MCP Server	Build a custom MCP server using the MCP Python SDK. Expose at least three distinct, functional tools. Each tool must perform a real operation on live database data: reads, writes, or queries. Tools must be clearly named and documented.
2	Pydantic AI Agent	Implement an agent using Pydantic AI that connects to your MCP server, invokes tools based on user intent, and returns structured, validated responses using Pydantic models.
3	Live Database	All agent-accessible data must reside in a proper database (SQLite minimum). The data must be dynamic; it should change as the agent takes write actions. No static files.
4	RBAC	Implement programmatic Role-Based Access Control. Define at least two distinct roles (e.g., admin, standard_user). Enforce restrictions at the tool level: certain tools may only be called by users holding the required role. Role violations must be caught and logged; never silently ignored.
5	Human-in-the-Loop	Certain tools must require explicit human approval before execution. The system must pause, present the pending action to the user, and wait for a confirmation or rejection signal. Rejections must abort the action and log the decision.
6	PII Masking	Before any data containing sensitive fields (names, emails, phone numbers, ID numbers, etc.) is passed to the LLM, those fields must be masked or redacted. Implement this as a reusable pre-processing layer, not inline string replacements.
7	Audit Logging	Produce a structured log of every tool call in the session. Each log entry must record: timestamp, user role, tool name, input parameters (PII-masked), outcome (success/failure), and any error details. Logs must be persisted to a file or database table.

8 Required Output Schemas

The following Pydantic models define the structured outputs your implementation may produce. Feel free to extend these schemas.

8.1 Tool Response Schema

```
from pydantic import BaseModel, Field
from typing import Any, Optional
from datetime import datetime

class ToolResponse(BaseModel):
    tool_name: str = Field(description='Name of the MCP tool that was invoked')
    success: bool = Field(description='Whether the tool executed successfully')
    result: Optional[Any] = Field(description='The tool output, PII-masked if applicable')
    error: Optional[str] = Field(description='Error message if success is False')
    requires_approval: bool = Field(description='Whether HITL approval was required')
    approved: Optional[bool] = Field(description='Human decision if approval was required')
```

8.2 Audit Log Entry Schema

```
class AuditEntry(BaseModel):
    timestamp: datetime = Field(description='UTC timestamp of the event')
    user_role: str = Field(description='Role of the user who triggered the action')
    tool_name: str = Field(description='Name of the MCP tool called')
    input_params: dict = Field(description='Tool inputs, with PII fields masked')
    outcome: str = Field(description='success | rejected_by_human | access_denied | error')
    error_detail: Optional[str] = Field(description='Stack trace or error message if applicable')
```

Note: You can decide to use Annotated from the typing library.

9 Step-by-Step Implementation Guide

Follow these steps in order. Each step builds on the previous one.

9.1 Set Up Your Project Environment

Initialise your environment and install required dependencies:

```
uv init
uv add pydantic-ai mcp
```

Recommended project structure (feel free to extend or modify it):

```

/action-agent
├── main.py                # Entry point
├── mcp_server/
│   ├── server.py         # MCP server definition
│   └── tools/             # Individual tool modules
├── agent/
│   ├── agent.py          # Pydantic AI agent
│   └── schemas.py        # Pydantic output models
├── security/
│   ├── rbac.py           # Role definitions and enforcement
│   └── pii_masker.py     # PII detection and masking
├── audit/
│   └── logger.py         # Structured audit logger
├── db/
│   ├── database.py       # DB connection and queries
│   └── seed.py           # Seed script for initial data
└── .env                  # Environment variables (excluded from git)

```

9.2 Build and Seed Your Database

Create your SQLite (or preferred) database and write a seed script that populates it with realistic sample data. Your data should contain at least one table with PII-like fields (names, emails, phone numbers) and one table representing resources or records that the agent will act upon.

Verify the database is live by running a query independently before connecting it to any tool.

9.3 Build the MCP Server

Create your MCP server and expose at least three tools. Each tool should:

- Have a clear, descriptive name and docstring (the LLM uses this to decide when to call it)
- Accept typed parameters validated by Pydantic
- Execute a real database operation. **Do not return hardcoded mock data.**
- Return a structured ToolResponse object

Example tool categories to consider:

- A read tool (e.g., look up a record, retrieve a list, check a status)
- A write tool that requires HITL approval (e.g., update a record, place an order, submit a request)
- An administrative tool restricted to elevated roles (e.g., delete a record, provision access, export data)

Start your server and confirm it is reachable before connecting the agent.

9.4 Implement RBAC (Least Privilege Access)

Define your role hierarchy and the permissions attached to each role. Implement a reusable access check function that:

- Accepts a user role and the name of the requested tool
- Returns True if the role is permitted to invoke that tool
- Returns False and generates an audit log entry if access is denied

Attach the check to every tool call. It must be executed before the tool logic runs, not after.

```
# Example RBAC definition
ROLE_PERMISSIONS = {
    'admin': ['get_record', 'update_record', 'delete_record', 'export_data'],
    'standard_user': ['get_record', 'submit_request'],
}

def check_access(role: str, tool_name: str) -> bool:
    permitted = ROLE_PERMISSIONS.get(role, [])
    return tool_name in permitted
```

9.5 Implement PII Masking

Write a masking function that detects and redacts PII fields in a dictionary before it is passed to the LLM. At minimum, mask:

- Email addresses
- Phone numbers
- National ID or passport numbers
- Any field whose key contains the words: name, email, phone, id_number, address, dob

Apply masking to all data retrieved from the database before it enters a prompt or a ToolResponse. Do not apply masking to data after the LLM has already seen it.

You can either use external libraries, or define your own custom functions (perhaps using regex matching).

```
# Example masking output
# Before: {'name': 'Jane Mwangi', 'email': 'jane@example.com', 'balance': 15000}
# After:  {'name': '**REDACTED**', 'email': '**REDACTED**', 'balance': 15000}
```

9.6 Implement the HITL Checkpoint

Identify which of your tools perform sensitive or irreversible actions. For each of these tools, insert a pause before execution:

- Display the pending action and its parameters to the user (with PII masked)
- Prompt the user to confirm or reject the action

- If confirmed, proceed with execution and log outcome: success
- If rejected, abort execution and log outcome: rejected_by_human
- Never proceed automatically, even if the user previously approved a similar action

9.7 Build the Pydantic AI Agent

Create your Pydantic AI agent and connect it to the MCP server. Configure the agent with:

- A system prompt that defines its role, capabilities, and constraints
- The MCP server as its tool source
- A result type using your structured ToolResponse schema

The agent's system prompt should instruct it to:

- Always check whether an action requires approval before proceeding
- Never expose raw PII in its responses
- Explain what it is doing and why at each step

9.8 Implement Audit Logging

Create a structured logger that records an AuditEntry for every tool call. Persist logs to a file (e.g., audit.log) and, optionally, to a database table for querying. Ensure:

- All PII in input parameters is masked before logging
- Errors include the full exception message and, where helpful, a stack trace
- Log entries are written immediately (not batched) so a crash does not lose evidence

10 Success Checklist

Your implementation is complete when all of the following are true:

	Criterion
☐	MCP server starts without errors and exposes at least 3 tools
☐	Pydantic AI agent successfully invokes tools via the MCP server
☐	All tool responses conform to the ToolResponse schema
☐	Tools read from and write to a live database (not static files)
☐	RBAC correctly permits access for authorised roles
☐	RBAC correctly denies access for unauthorised roles and logs the violation
☐	HITL checkpoint pauses and awaits human confirmation for sensitive tools
☐	Rejected HITL actions are aborted and logged correctly
☐	PII fields are masked before reaching the LLM or appearing in logs
☐	Audit log contains a complete, structured entry for every tool call

- ☐
The system runs end-to-end from the terminal with no unhandled exceptions

11 Stretch Goals (Optional)

Once your core implementation is working, extend it with one or more of the following. Stretch goals do not replace mandatory requirements, they demonstrate additional depth.

Third-Party Integration

Extend your MCP server by integrating an external API as an additional tool. Examples:

- A weather API (Open-Meteo is free, no API key required) to surface environmental context
- A publicly available MCP server (browse the MCP registry at modelcontextprotocol.io/registry) to complement your custom tools
- A notification API (e.g., Resend for email) triggered after a HITL-approved action

Role Escalation

Implement a temporary privilege escalation mechanism: a `standard_user` may request elevated access for a specific tool, which triggers a HITL approval workflow. If approved, the elevated permission is granted for the current session only and logged with a reason.

Persistent Audit Dashboard

Store audit logs in a database table and add a simple query interface (a CLI command or a minimal Streamlit page) that lets a reviewer filter logs by role, tool, outcome, or time range.

Structured Error Recovery

Implement a retry mechanism: if a tool call fails due to a transient error (e.g., database lock), the agent automatically retries up to a configurable maximum. Each retry is logged separately. After the maximum is exceeded, the error is escalated to a HITL checkpoint rather than failing silently.

12 Evaluation Criteria

Criteria	Weight	Description
MCP Server & Tool Quality	25%	Are tools well-named, correctly documented, and backed by live data? Do they handle errors gracefully?
RBAC Implementation	20%	Are roles correctly defined and enforced at the tool level? Are violations caught and logged?
HITL Checkpoint	20%	Does the system genuinely pause for approval? Are both confirm and reject paths handled correctly?
PII Masking	15%	Is PII masked before reaching the LLM? Is the masking layer reusable and applied consistently?

Audit Logging	15%	Is the audit log complete, structured, and persisted? Does it capture all required fields?
Code Quality & Structure	5%	Is the codebase modular, readable, and well-organised? Is the README clear?

13 Reference Documentation

Core Documentation

Resource	URL
Pydantic AI	ai.pydantic.dev
MCP Python SDK	modelcontextprotocol.io/docs/tools/python
Pydantic (Validation)	docs.pydantic.dev
uv Package Manager	docs.astral.sh/uv
MCP Registry	modelcontextprotocol.io/registry

Recommended Reading Areas

- **Pydantic AI:** Agents, Tool Calling, Result Types, Dependency Injection, Structured Outputs
- **MCP Python SDK:** Server Setup, Tool Definition, Client Connection, Transport Options
- **Pydantic:** BaseModel, Field, Validators, model_validator

14 Why Documentation Matters

Agentic Engineering is a rapidly evolving field. The MCP specification, Pydantic AI's tool-calling interface, and LLM provider APIs are updated frequently. Engineers who memorise APIs rather than learning to read documentation are continuously left behind.

Every framework you use in this course will change. What will not change is your ability to open a documentation page, locate the right section, read an example, and apply it to your problem.

Develop this habit now. When something does not work, check the official documentation before asking for help. You will find the answer faster, and you will retain it longer.

15**Instructor Notes**

This module marks the first time your agent can change the world; it can write to a database, trigger processes, and produce real side effects. The governance components (RBAC, HITL, PII masking, audit logging) are not optional extras. They are what separate a prototype from a production-grade system.

You may ask why governance is introduced so early in the course. The answer is intentional: it is far easier to build governance in from the beginning than to retrofit it onto a system that was designed without it. Every professional agentic system you build from this point forward will need these controls. Start treating them as defaults, not additions. Don't be worried about it though, we'll learn more about AI Agent Governance later in the course!

[Build systems you are proud to put your name on.](#)